

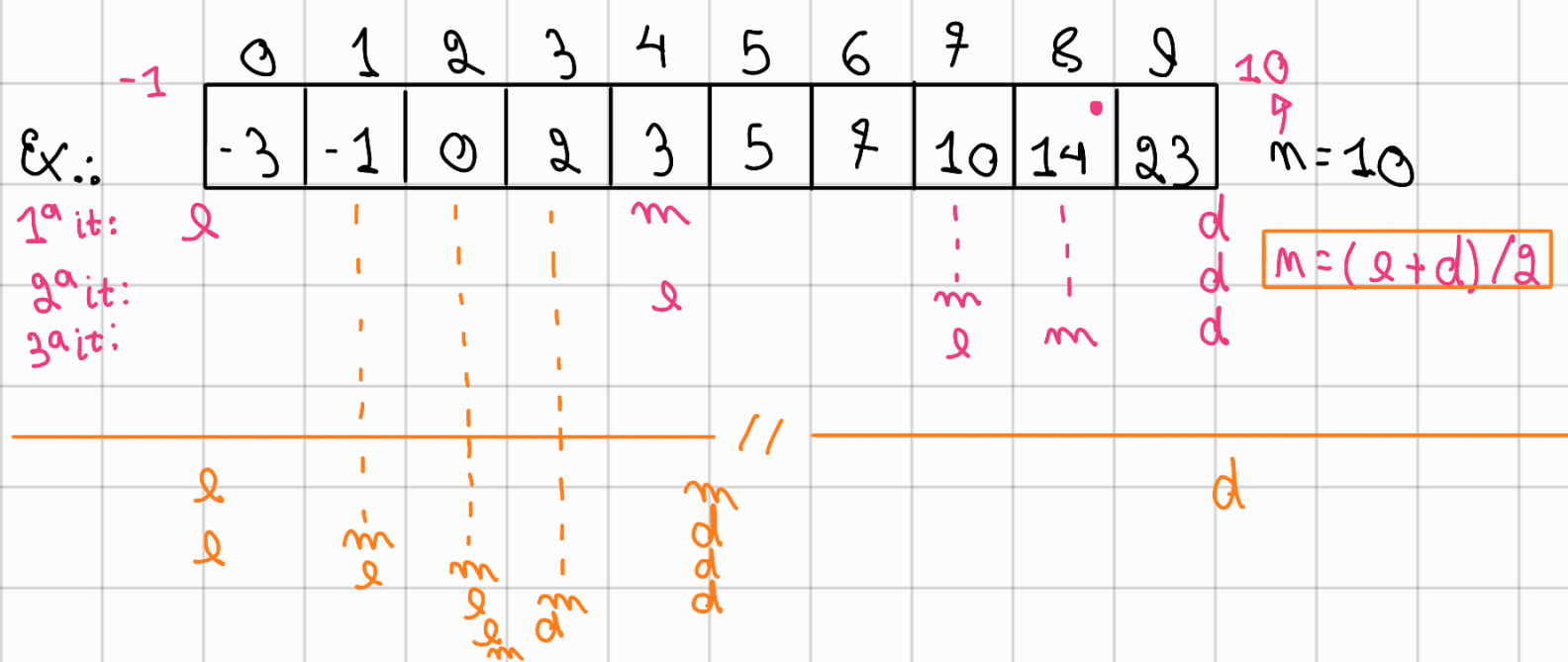
Busca

Suponha que temos um conjunto S de elementos. O problema de base consiste em determinar se $x \in S$, para um dado x . Se S for representado como um vetor, podemos também querer saber que posição x ocupa no vetor.

```
int BUSCA(int *v, int n, int x){  
    for(int i=0; i<n; i++){  
        if(v[i] == x) return i;  
    }  
    return -1;  
}
```

Complexidade: $O(n)$.

Uma versão melhor teríamos com um vetor ordenado



```

int busca(int *v, int n, int x){
    int e = -1, d = n;
    int m = (e+d)/2;
    while(e < d-1){
        if (v[m] == x) return m;
        if (v[m] > x) d = m;
        else e = m;
        m = (e+d)/2;
    }
    return d;
}

```

Problema da busca num vetor ordenado:

Dado $v[0 \dots n-1]$ um vetor ordenado (em ordem crescente) e um elemento x , queremos encontrar j tal que

$$v[j-1] < x \leq v[j].$$

De seja: encontrarmos j tal que $v[j] = x$ ou tal que x deveria estar na posição j para que o vetor permanecesse ordenado.

Este algoritmo é chamado de **busca binária**.

Desempenho

O algoritmo de BB itera k vezes, de tal forma a procurar num vetor de tamanho n , depois $\frac{n}{2}$, $\frac{n}{2^2}$, $\frac{n}{2^3}$, ..., $\frac{n}{2^k} < 1 \Rightarrow n < 2^k \therefore k > \lg n$

Portanto, o custo da BB é $O(\lg n)$

```
int bb(int *v, int e, int d, int x) {  
    if (e < d - 1) {  
        int m = (e + d) / 2;  
        if (v[m] >= x) RETURN bb(v, e, m, x);  
        ELSE RETURN bb(v, m, d, x);  
    }  
    RETURN d;  
}
```

```
int BUSCA(int *v, int n, int x) {  
    RETURN bb(v, -1, n, x);  
}
```